

DOSSIER TÉCNICO

CONSUMO DE APIS EN ARQUITECTURAS FRONTEND CON IONIC Y ANGULAR

Módulo: Desarrollo de Interfaces (0488) | **Ciclo:** CFGS Desarrollo de Aplicaciones Multiplataforma (DAM)

Alineación Normativa (Comunidad de Madrid):

- **RA 1:** Confecciona interfaces de usuario analizando las arquitecturas de las aplicaciones y sus componentes.
 - **CE 1.a:** Se han analizado los distintos tipos de arquitecturas de aplicaciones.
 - **CE 1.e:** Se han diferenciado las responsabilidades de cada uno de los componentes de la arquitectura (separación cliente-servidor).
- **RA 3:** Crea componentes visuales valorando y empleando herramientas específicas.
 - **CE 3.b:** Se han creado controladores de eventos (gestión asíncrona de peticiones HTTP).
 - **CE 3.d:** Se han enlazado los componentes a orígenes de datos.

1. BASE TEÓRICA TÉCNICA

1.1. ¿Qué es una API?

Una **API (Application Programming Interface)** es un conjunto de reglas, protocolos y herramientas que permite a diferentes aplicaciones de software comunicarse entre sí. Actúa como un contrato o intermediario abstracto: el sistema cliente solicita un recurso o acción especificada, y el sistema proveedor entrega la respuesta sin que el cliente necesite conocer la implementación interna del sistema servidor.

1.2. Importancia en el Desarrollo Full Stack

En las arquitecturas modernas, el desarrollo no se concibe como un bloque monolítico. La integración de APIs en el ecosistema Full Stack proporciona ventajas arquitectónicas críticas:

- **Desacoplamiento (Separation of Concerns):** El frontend (Ionic/Angular) y el backend (Node, Spring, Django) se desarrollan, escalan y despliegan de forma independiente.
- **Reusabilidad Omnicanal:** Una única API centralizada puede servir datos simultáneamente a una aplicación web (React/Angular), una app móvil (Ionic/Flutter) y sistemas de terceros.
- **Escalabilidad:** Permite balancear la carga y escalar los servicios del backend sin alterar el código de la interfaz de usuario.
- **Seguridad:** La lógica de negocio y las credenciales de la base de datos permanecen ocultas en el servidor. La interfaz solo interactúa con endpoints securizados (mediante JWT, OAuth).

1.3. Tipos de APIs (Por Arquitectura)

- **REST (Representational State Transfer):** Basada en el protocolo HTTP, utiliza verbos estándar (GET, POST, PUT, DELETE) y URIs para manipular recursos. Es el estándar de facto actual.
- **GraphQL:** Desarrollada por Facebook, permite a los clientes consultar de forma precisa y

estricta la estructura de datos que necesitan, evitando el *over-fetching* (traer datos de más) y el *under-fetching* (traer datos de menos).

- **SOAP (Simple Object Access Protocol):** Protocolo basado en XML. Altamente estructurado, tipado y seguro, pero más pesado y verboso. Común en entornos bancarios o legacy.
- **gRPC:** Basado en HTTP/2 y Protocol Buffers. Diseñado por Google para comunicación ultrarrápida entre microservicios.

1.4. ¿Qué es una API REST?

REST es un estilo arquitectónico para sistemas distribuidos. Una API es considerada REST si cumple las siguientes restricciones:

1. **Arquitectura Cliente-Servidor:** El cliente gestiona el UI/UX; el servidor procesa datos y almacenamiento.
2. **Stateless (Sin Estado):** Cada petición HTTP del cliente contiene toda la información necesaria para procesarla. El servidor no almacena sesiones del cliente.
3. **Caché:** Las respuestas deben indicar explícitamente si pueden ser almacenadas en caché por el cliente.
4. **Interfaz Uniforme:** Los recursos se identifican mediante URIs lógicas (ej. `/api/usuarios/123`), y se manipulan mediante representaciones estándar (JSON, XML).

1.5. Diferencia entre REST y RESTful

Aunque a menudo se usan como sinónimos, técnicamente existe un matiz conceptual:

- **REST:** Es el **paradigma** o conjunto de reglas y restricciones arquitectónicas definidas por Roy Fielding.
- **RESTful:** Es el **adjetivo** aplicado a un servicio web o aplicación específica que implementa y cumple estrictamente con los principios de la arquitectura REST (ej: "Hemos desplegado un servicio RESTful en Node.js").

2. EJEMPLO DE CÓDIGO FUNCIONAL: CONSUMO DESDE IONIC-ANGULAR

El siguiente ejemplo implementa una arquitectura basada en servicios en Angular para consumir una API REST pública (JSONPlaceholder), mostrando la lista de usuarios en la vista de Ionic.

Requisitos previos: Importar `provideHttpClient` en `main.ts` o `HttpClientModule` en `app.module.ts`.

Paso 1: Definición de la Interfaz (Modelo de Datos)

Creamos una interfaz TypeScript para tipar los datos entrantes, garantizando consistencia en el diseño de la UI.

`src/app/interfaces/user.interface.ts`

```
export interface User {  
  id: number;  
  name: string;  
  email: string;  
  phone: string;  
  website: string;  
}
```

Paso 2: Creación del Servicio de Conexión (Lógica de Negocio)

Aislamos la lógica HTTP en un servicio inyectable. Nunca se deben hacer peticiones HTTP directamente desde el componente.

`src/app/services/api.service.ts`

```
import { Injectable, inject } from '@angular/core';  
import { HttpClient } from '@angular/common/http';  
import { Observable } from 'rxjs';  
import { User } from '../interfaces/user.interface';  
  
@Injectable({  
  providedIn: 'root'  
})  
export class ApiService {  
  private readonly baseUrl: string = 'https://jsonplaceholder.typicode.com/users';  
  
  // Inyección de dependencias moderna en Angular  
  private http = inject(HttpClient);  
  
  /**  
   * Obtiene la lista completa de usuarios (Método GET)   */
```

```

    */
    getUsers(): Observable<User[]> {
        return this.http.get<User[]>(this.baseUrl);
    }
}

```

Paso 3: Controlador del Componente (Control de UI)

src/app/home/home.page.ts

```

import { Component, OnInit, inject } from '@angular/core';
import { ApiService } from '../services/api.service';
import { User } from '../interfaces/user.interface';
import { Observable } from 'rxjs';
import { IonicModule } from '@ionic/angular';
import { CommonModule } from '@angular/common';

@Component({
    selector: 'app-home',
    templateUrl: 'home.page.html',
    styleUrls: ['home.page.scss'],
    standalone: true,
    imports: [IonicModule, CommonModule],
})
export class HomePage implements OnInit {
    // Manejo reactivo del estado
    public users$: Observable<User[]>;
    private apiService = inject(ApiService);

    ngOnInit() {
        this.fetchData();
    }

    fetchData() {
        // Asignamos el Observable directamente en lugar de suscribirnos manualmente
        // Delegamos la subscripción al AsyncPipe en la vista
        this.users$ = this.apiService.getUsers();
    }
}

```

Paso 4: Vista Declarativa (HTML)

Utilizamos componentes nativos de Ionic y el pipe asíncrono (async) de Angular para gestionar automáticamente la suscripción y evitar fugas de memoria (memory leaks).

src/app/home/home.page.html

```
<ion-header>
  <ion-toolbar color="primary">
    <ion-title>Directorio de Usuarios</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content>
  <!-- Renderizado condicional mientras se resuelve la petición asíncrona -->
  <ion-list *ngIf="users$ | async as users; else loading">
    <ion-item *ngFor="let user of users">
      <ion-avatar slot="start">
        <!-- Avatar generado dinámicamente con el nombre del usuario -->
        <img [src]="https://ui-avatars.com/api/?name=' + user.name" alt="Avatar"/>
      </ion-avatar>
      <ion-label>
        <h2>{{ user.name }}</h2>
        <p>{{ user.email }}</p>
        <p><ion-icon name="globe-outline"></ion-icon> {{ user.website }}</p>
      </ion-label>
    </ion-item>
  </ion-list>

  <!-- Template local para el estado de carga (Skeleton Text) -->
  <ng-template #loading>
    <ion-list>
      <ion-item *ngFor="let i of [1,2,3,4,5]">
        <ion-avatar slot="start"><ion-skeleton-text animated></ion-skeleton-text></ion-avatar>
        <ion-label>
          <h2><ion-skeleton-text animated style="width: 50%"></ion-skeleton-text></h2>
          <p><ion-skeleton-text animated style="width: 80%"></ion-skeleton-text></p>
        </ion-label>
      </ion-item>
    </ion-list>
  </ng-template>
</ion-content>
```

3. EJERCICIO PROPUESTO PARA EL ALUMNO

Título: Sistema Maestro-Detalle con la Rick and Morty API.

Objetivo: Desarrollar una aplicación en Ionic que consuma una API REST pública paginada e implemente un patrón de diseño maestro-detalle (Master-Detail).

Requerimientos:

1. **Conexión y Tipado:** Crear interfaces TypeScript estrictas para analizar las respuestas de https://rickandmortyapi.com/api/character
2. **Servicio Centralizado:** Implementar un CharacterService que exponga métodos para recuperar la lista de personajes (GET /character) y los detalles de un personaje específico (GET /character/{id}).
3. **Vista Maestra:** Implementar un listado (<ion-list>) mostrando el avatar, nombre y especie de los personajes.
4. **Navegación:** Al pulsar sobre un elemento del listado, navegar (Router) a una vista de detalles pasando el id por parámetro.
5. **Vista Detalle:** Mostrar información ampliada (origen, estado de vida, género) usando tarjetas visuales (<ion-card>).
6. **Control de Errores (Avanzado):** Gestionar las posibles respuestas HTTP de error (ej: error de red, 404) mediante operadores RxJS (catchError) o un interceptor, mostrando un <ion-toast> al usuario.